

DIT343: PROJECT: WEB ENGINEERING – ASSIGNMENT/SCHEDULE

University of Gothenburg

Deadline Final Release: **Wed Jan 07 08:59**

General Remarks

- **Self- and Peer Assessment:** As an individual part of every milestone, *every* group member shall reflect on the contribution of the individual group members. In case we suspect that individuals have contributed significantly less than others, we might ask questions (email/interview) and reserve to remove individuals from a group.
Deliverable: A filled in peer evaluation questionnaire provided and submitted on Canvas.
- **Flexible project topics:** The requirements for the assignment topic are formulated flexible enough to encourage you to come up with an idea that you find motivating. At the same time, define a project that fulfills at least these requirements. Plan ahead!
- **Plagiarism:** Plagiarism of any form will yield an immediate fail on the project, and further disciplinary steps might be taken. Do not copy any projects from peers or from the internet - even if slightly modified!
- **Questions:** Ask in Slack if anything is unclear!
- **Commit your code:** We use your commits in GitLab as an indicator of your contribution to the project. If another person commits your code, we do not consider it as your contribution. Thus, your team members should not commit code that you have produced!
- **Policies for the use of AI:** The goal of the course is to learn web development. AI can help with this if it is used in the right way, but it can also undermine the learning effect. Therefore, we have defined a set of policies regarding the use of AI in this course that you need to conform to (see Section 9).

1 Important Deadlines

Deadline	Group Submission	Individual Submission
Fri Nov 07 09:00	Milestone 0 (MS0): System Definition ¹	None
Mon Nov 24 23:59	Milestone 1 (MS1): Backend ²	Self and Peer Assessment MS1 ³
Fri Dec 05 09:00	Milestone 2 (MS2): Frontend ⁴	Self and Peer Assessment MS2 ⁵
Fri Dec 19 09:00	Milestone 3 (MS3): Responsive Frontend ⁶	Self and Peer Assessment MS3 ⁷
Wed Jan 07 08:59	Release: Hard Final Submission Deadline ⁸	None

¹<https://canvas.gu.se/courses/92033/assignments/349168>

²<https://canvas.gu.se/courses/92033/assignments/349169>

³<https://canvas.gu.se/courses/92033/quizzes/70177>

⁴<https://canvas.gu.se/courses/92033/assignments/349170>

2 Overall Project Description

The group project assignments consist of four parts that must be integrated and presented as your final product at Release. In part 0, you will define your project. In part 1, you will implement the backend (i.e., server). You will demonstrate your working backend web client in a follow-up meeting to your TA to complete MS1. In part 2, you will implement the frontend (i.e., client) that connects to your backend from part 1. You will demonstrate your working frontend and backend in a follow-up meeting to your TA to complete MS2. In part 3, you will extend the frontend to support responsive web design by automatically adjusting for different screen sizes and prepare a presentation (live demo). You will demonstrate your working responsive frontend integrated with your working backend to your TA to complete MS3. This serves as "dress rehearsal" for the final release presentation.

Release: You will demonstrate your complete working system *to your examiner* in the final release presentation (as described in Section 8). Your release deliverables and presentation will determine your assignment grade.

3 Grading

The exact purpose and functionality of the application will be decided by each group individually, following a number of requirements described in the following sections. For a grade of 3, the application should be in working order, and fulfill **all** the fundamental requirements. For a grade of 4, the application should function well, fulfilling **all** fundamental and extended requirements. For a grade of 5, the application should function excellently, fulfilling **all** requirements – *fundamental*, *extended*, and *advanced* – while also expanding on the functionality. This can include integrating external APIs. The key aspect is to demonstrate the ability to independently apply the concepts described in the course, thus, going beyond the requirements, rather than doing more of the same. For example, only adding more entities would be insufficient. A proposal for this functionality needs to be approved by the examiner to qualify for a 5. Your proposal should contain a summary of the added functionality. As well as a brief description of the necessary **backend and frontend** enhancements to achieve this functionality. It must be clear from the description that implementing this feature requires a deeper understanding of web development and does not simply consist of adding additional entities.

4 Milestone 0 (MS0): System Definition

The topic of this milestone is: **Defining your project**.

You will start by defining the purpose of the overall system – backend and frontend together – which you will implement in the project of this course. The system purpose needs to fulfill the following requirements:

4.1 Fundamental

For a grade of 3, the system needs to meet all the requirements below.

⁵<https://canvas.gu.se/courses/92033/quizzes/70178>

⁶<https://canvas.gu.se/courses/92033/assignments/349172>

⁷<https://canvas.gu.se/courses/92033/quizzes/70179>

⁸<https://canvas.gu.se/courses/92033/assignments/349173>

1. The system shall provide sensible CRUD (Create, Read, Update, Delete) functionality on *at least four entities*.
2. The system shall serve a unified purpose (e.g., task management, issue tracking). In other words: "your overall system should make sense together". Counterexample: It is not ok to use completely unrelated entities.
3. The system shall be of a scope that is realistically implementable during the 7-week course period.

4.2 Extended, Advanced

No further requirements for MS0. However, if you are aiming for grade 5, you may start brainstorming about advanced features.

Deliverable: An (a) entity relationship (ER) diagram⁹ of your system with a (b) short textual description of the purpose and pages following the template in README.md¹⁰. Commit your diagram as *PNG* under `images/er_diagram.png` and the description in the README.md file within your "-web" repository¹¹ and then submit a GitLab link to the latest commit in Canvas.

Example link: <https://git.chalmers.se/courses/dit341/group-00-web/commit/7d5735b900a2435a3567dcf2881c1f168b6e5a90>
where `7d5735b9...` is the full SHA hash of your commit copied from *Repository > Commits*.

5 Milestone 1 (MS1): Backend Implementation

The topic of this milestone is: **Implementing the backend**.

You will implement the backend as a RESTful JSON API written in JavaScript based on Node.js and with MongoDB persistency using the Object Data Model (ODM) of Mongoose. The backend code needs to fulfill the following requirements:

5.1 Fundamental

For a grade of 3, the system needs to meet all the requirements below.

1. The CRUD functionality described in the system description through (a) *entities* and (b) *relationships* shall be made accessible through a RESTful JSON API, as described in the backend lectures (L3 and L4).
2. Every of the following CRUD endpoints shall be demonstrated *at least twice* (using the resource `camels` as example here):
 - (a) `POST /camels`
 - (b) `GET /camels`
 - (c) `DELETE /camels`
 - (d) `GET /camels/:id`
 - (e) `PUT /camels/:id`

⁹To create the diagram, we recommended the free diagram editor <https://www.draw.io/>

¹⁰<https://git.chalmers.se/courses/dit342/group-00-web>

¹¹There are placeholder files you should simply replace or edit with your content

- (f) PATCH /camels/:id
 - (g) DELETE /camels/:id
3. The API shall expose at least the following endpoints for *at least one* relationship between two entities according to the relationship's semantics (to its meaning)¹²:
 - (a) POST /cars/:car_id/drivers
 - (b) GET /cars/:car_id/drivers
 - (c) GET /cars/:car_id/drivers/:driver_id
 - (d) DELETE /cars/:car_id/drivers/:driver_id
 4. The implementation shall follow the practices described in the respective material provided in Canvas:
 - (a) The CRUD functionality must follow the discussed naming conventions (e.g., plural nouns).
 - (b) The CRUD functionality must follow the discussed behavior semantics (i.e., safe, unsafe, idempotent).
 5. All the endpoints must work as intended, demonstrated through a collection of Postman requests executable via `npm test`. Every endpoint must have *at least one* Postman request¹³ demonstrating its use.
 6. There shall not be any (unhandled) exceptions or crashes (e.g., due to malformed input data).
 7. The backend code must be valid Javascript, install via `npm install`, and run via `npm start`. Your implementation shall be based on the provided template <https://git.chalmers.se/courses/dit342/group-00-web>.

5.2 Extended

For a grade of 4, in addition to the Fundamental requirements above, the system needs to meet all the requirements below.

1. At least one collection endpoint shall provide *at least two* of the following functionalities: filtering, sorting, field selection, or pagination.
2. All endpoints shall use appropriate HTTP status codes (e.g., for handling errors).
3. At least one endpoint shall implement HTTP method overriding.
4. All the endpoints must work as intended, demonstrated through a collection of **meaningful** Postman¹⁴ requests executable via `npm test`. Every endpoint must have *at least a handful of tests which test correct and incorrect (faulty) requests*. Tests should validate whether the right status codes are returned, but (especially for GET requests) also whether the endpoint returns JSON documents with the right format and content. All tests must pass in the Chalmers GitLab continuous integration (CI) pipeline¹⁵.

¹²Hint: The populate functionality of mongoose might be helpful: <https://mongoosejs.com/docs/populate.html>

¹³<https://learning.postman.com/docs/postman/sending-api-requests/requests/>

¹⁴<https://learning.postman.com/docs/postman/sending-api-requests/requests/>

¹⁵Make sure you commit regularly and test CI early enough as builds can take a while to run.

5.3 Advanced

For a grade of 5, in addition to the Fundamental and Extended requirements above, the system needs to meet all the requirements below.

1. The system shall provide functionality going beyond the requirements, as outlined in Section 3. This functionality shall tie in with the system purpose, see MS0 Requirement 2. A proposal needs to be included in the deliverable. The proposal will be reviewed by the examiner and needs to be approved to qualify for a grade of 5. Your proposal should contain a summary of the added functionality. As well as a brief description of the necessary **backend and frontend** enhancements to achieve this functionality. It must be clear from the description that implementing this feature requires a deeper understanding of web development and does not simply consist of adding additional entities.
2. The advanced functionality should involve the backend.
3. Implement API versioning.
4. At least one endpoint shall implement HATEOAS.

Deliverable: Revisit the project purpose and scope description and check that it's up-to-date with the implementation. Create a new release tag in GitLab under *Repository > Tags* called *milestone1*¹⁶ and then submit a GitLab link to the *tagged commit* on Canvas.

- Example link: <https://git.chalmers.se/courses/dit341/group-00-web/commit/7d5735b900a2435a3567dcf2881cf168b6e5a90> where `7d5735b9...` is the full SHA hash of your commit copied from *Repository > Commits*.
- Ensure that all your changes are committed and the GitLab pipeline succeeded before you tag your code for submission.
- **Important:** Never ever force push to *master*! This can ruin all your colleagues' references and invalidate all your milestone submissions. GitLab enforces this policy if you keep your master branch protected.

6 Milestone 2 (MS2): Frontend Implementation

The topic of this milestone is: **Implementing the frontend.**

You will add a web frontend written in HTML, CSS, and JavaScript based on Vue.js, Axios, and Bootstrap that interacts with your backend via its RESTful JSON API endpoints. The frontend code needs to fulfill the following requirements:

6.1 Fundamental

For a grade of 3, the system needs to meet all the requirements below.

1. The frontend needs to provide sensible CRUD access to (a) *at least four* entities and (b) *at least one* relationship, using the RESTful API implemented in MS1.
2. Every of the following CRUD endpoints shall be used *at least once* in your frontend (using the resource `camels` as example here):

¹⁶<https://stackoverflow.com/questions/29520905/how-to-create-releases-in-gitlab>

- (a) POST /camels
 - (b) GET /camels
 - (c) DELETE /camels
 - (d) GET /camels/:id
 - (e) PUT /camels/:id
 - (f) PATCH /camels/:id
 - (g) DELETE /camels/:id
3. The frontend shall use at least the following endpoints for *at least one* relationship between two entities according to the relationship's semantics (i.e., to its meaning). Hence, the frontend shall provide forms, buttons etc to create and query objects through a relationship:
- (a) POST /cars/:car_id/drivers
 - (b) GET /cars/:car_id/drivers
4. The frontend code must be valid Javascript and HTML5 and be served via `npm run serve`. Your implementation shall be based on the provided template <https://git.chalmers.se/courses/dit342/group-00-web>.

6.2 Extended

For a grade of 4, in addition to the Fundamental requirements above, the system needs to meet all the requirements below.

1. Your web application (backend and frontend) must run in production mode, either locally¹⁷ or deployed to a cloud service provider.
2. The frontend shall use the filtering, sorting, field selection, or pagination of the RESTful API implemented in MS1.

6.3 Advanced

For a grade of 5, in addition to the Fundamental and Extended requirements above, the system needs to meet all the requirements below.

1. The advanced functionality (MS1 Requirement 5.3.1) should involve the frontend.
2. The frontend shall use the HATEOAS functionality of the RESTful API implemented in MS1.

Deliverable: Revisit the project purpose and scope description and check that it's up-to-date with the implementation. Create a new release tag in GitLab under *Repository > Tags* called milestone2¹⁸ and then submit a GitLab link to the *tagged commit* on Canvas.

¹⁷https://git.chalmers.se/courses/dit342/group-00-web/blob/master/docs/LOCAL_DEPLOYMENT.md

¹⁸<https://stackoverflow.com/questions/29520905/how-to-create-releases-in-gitlab>

7 Milestone 3 (MS3): Responsive Web Design

The topic of this milestone is: **Making the frontend responsive.**

You will extend the frontend to support responsive web design by automatically adjusting for different screen sizes.

7.1 Fundamental

For a grade of 3, the system needs to meet all the requirements below.

1. The frontend shall implement a reasonable and convenient user interface (UI). For example: It shall provide forms to enter or edit data fields.
2. On *all* pages, the frontend shall be responsive (i.e., content must scale appropriately for small devices, not hiding content) for at least the following two cases using responsive classes with Bootstrap:
 - (a) Devices with a screen width $\geq 768px$ (*medium, large, and extra large* devices in Bootstrap).
 - (b) Devices with a screen width $< 768px$ (*small and extra small* devices in Bootstrap).
3. On *at least one* page, the frontend shall provide a different layout for at least the two cases described above using *different* responsive classes with Bootstrap.
4. There shall not be any (unhandled) exceptions or crashes (e.g., if the backend is unreachable).

7.2 Extended

For a grade of 4, in addition to the Fundamental requirements above, the system needs to meet all the requirements below.

1. All styling and layout shall be consistent across all pages. Your web application shall have a polished UI, giving a thorough, coherent impression.
2. All styling and layout shall be done solely through CSS (and Bootstrap). Tables should only be used to display lists of content, and not as a hack for layout.
3. No technical details such as database identifiers shall be displayed.

7.3 Advanced

For a grade of 5, in addition to the Fundamental and Extended requirements above, the system needs to meet all the requirements below.

1. The advanced functionality (MS1 Requirement 5.3.1) shall be implemented according to the approved proposal.

Deliverable: Revisit the project purpose and scope description and check that it's up-to-date with the implementation. Create a new release tag in GitLab under *Repository > Tags* called milestone3¹⁹ and then submit a GitLab link to the *tagged commit* on Canvas.

¹⁹<https://stackoverflow.com/questions/29520905/how-to-create-releases-in-gitlab>

8 Release: Hard Final Submission Deadline

Live Presentation

You will present and demonstrate to your examiner that your responsive web application (i.e., frontend) and backend fulfill the cumulative requirements of all milestones (MS1, MS2, MS3) in a max **8** minute presentation. Your group will be assigned an individual presentation slot. The presentation needs to fulfill the following requirements:

1. You need to demonstrate *live* that your web application fully works (i.e., convince your examiner that your system fulfills the requirements). Show your application running, please do not use any slides:
 - (a) Use a web browser to demonstrate your frontend. Show interesting frontend code where appropriate.
 - (b) Use the device mode²⁰ of your web browser development tools²¹ to demonstrate your responsive design (e.g., choosing iPhone 8). Show interesting responsive frontend code where appropriate.
 - (c) Use Postman to demonstrate requests against your backend. Show interesting backend code where appropriate.
 - (d) Your web application (backend and frontend) must run in production mode, either locally or deployed to a cloud service provider.
2. Every group member needs to present one part and be able to answer questions. These can include explaining technical aspects of how your application works, as well as reflections on what worked well and what you would change.

Important: You have strictly limited presentation time and therefore prepare, test, and *practise* your presentation beforehand!

Deliverable:

1. *One* single teaser slide with your (a) best *max four* screenshots including desktop and mobile screen size, (b) a *single* sentence describing the purpose of your system and (c) your group name (e.g., AG7) and all group member names. Export your slide as *PNG* image and commit it into your "-web" repository under `images/teaser.png`.
2. Use the release tag name `release1` for the final release (instead of `milestone3`).
3. You do not need to submit a Self- and Peer Assessment form again for the final release.

9 Policies for the use of AI

The policies for the use of AI set out below apply to the entire duration of the project course.

²⁰<https://developer.chrome.com/docs/devtools/device-mode>

²¹<https://developers.google.com/web/tools/chrome-devtools>

	Summary	Policy	Details
R1	Use AI autocompletion or code suggestions, for instance via GitHub Copilot	Allowed	Modern IDEs have LLM-powered autocompletion features. Using them to help with coding or exploring the APIs of the libraries we are using in the project is allowed. Similarly, asking tools such as ChatGPT to generate examples of how to, e.g., make a call using Axios, is allowed.
R2	Vibe coding	Not Allowed	Despite R1, you are still expected to be the primary author of your code. Do not "vibe code", i.e., let an AI generate most or all of your project code. The goal of the course is for you to learn how to develop web applications, not find out if an LLM could do it (in the small scale of a university course).
R3	Use app generators such as Lovable or Replit.	Not Allowed	Similar to R2, the usage of tools that attempt to generate entire apps for you is not allowed. You have to learn how to architect and build web applications yourself. Also, note that the usage of these tools is often rather noticeable. If it becomes evident that you are working with an app generator, we reserve the right to ask you to delete your current project and restart with a new application.
R4	Use of agent-based coding systems, such as gemini-cli.	Not Suggested	Systems such as gemini-cli can be used in a wide range of ways, from a small helper for tricky issues to basically a vibe coding platform. At this stage of your education, we do not recommend using them , but you can as long as you ensure that it's still mostly you coding (e.g., what you use them for is more similar to R1 than R2).
R5	Use AI to debug or understand code.	Allowed, but beware	It may be useful to ask for AI help for debugging an issue. However, make sure that you still understand the fixes proposed by the AI and validate that they actually fix your issue (and why). In our experience, many AI systems have a tendency to hide issues rather than resolve them (i.e., suppress an error or disarm a failing test case).
R6	Generate tests or test cases using AI	Allowed, but beware	Generating tests at scale (both unit tests but also Postman/endpoint tests) may be a decent use for AI in the course. However, make sure that the tests make sense , i.e., that they validate important properties of the system and that they fail when the system is not working. This year, we will assess more strictly than in previous years that Postman tests do not just "exist", but that actual thought went into them.
R7	Generate documentation, commit messages, or other auxiliary artifacts.	Allowed	It's fine (in this course!) to use AI to help with or generate non-code artifacts.
R8	Use AI as a sort of TA	Allowed	A good use for an AI such as ChatGPT is as a "TA" of sorts, which you can ask for further explanations, feedback on your code or design, or to define concepts you do not understand. However, beware of hallucinations (and of course any information you receive from the LLM is not authoritative, we still reserve the right to criticise your code even if ChatGPT says that it's cool). Further, avoid the temptation to immediately ask ChatGPT for help whenever you are slightly stuck. Learning requires struggle , and immediate access to help 24/7 is not always a good thing for your learning and retaining the material..

Summary:

1. You are supposed to learn how to develop Web applications yourself. Using AI that allows you to generate code that you don't really understand is not allowed.
2. You are still responsible for your code. You need to understand what it is doing and what the consequences are.
3. You can use AI for repetitive tasks (e.g., generating many tests or documentation).